

HiconAgent: History Context-aware Policy Optimization for GUI Agents

Xurui Zhou¹, Gongwei Chen¹, Yuquan Xie¹, Zaijing Li¹, Kaiwen Zhou³,
Shuai Wang³, Shuo Yang¹, Zhuotao Tian¹, Rui Shao^{1,2†}

¹Harbin Institute of Technology, Shenzhen ²Shenzhen Loop Area Institute ³Huawei Noah's Ark Lab

zhouxurui1314@gmail.com chengongwei@hit.edu.cn shaorui@hit.edu.cn

<https://github.com/JiuTian-VL/HiconAgent>

Abstract

Graphical User Interface (GUI) agents require effective utilization of historical context to perform sequential navigation tasks. While incorporating past actions and observations can significantly improve decision-making, naively using full history leads to excessive computational overhead and potential distraction from irrelevant information. In this work, we introduce **HiconAgent**, a GUI agent trained with **History Context-aware Policy Optimization (HCPO)** for effective and efficient utilization of historical information. HCPO explicitly optimizes history usage in both sampling and policy updates by integrating two complementary components: (1) **Dynamic Context Sampling (DCS)** presents the agent with variable-length histories during sampling, enabling adaptive use of the most relevant historical context to improve sequential decision quality; (2) **Anchor-guided History Compression (AHC)** refines the policy update phase via a dual-branch optimization strategy, where the compressed branch drops history observations while keeping history actions as information flow anchors. The compressed and uncompressed branches are coupled through a history-enhanced alignment loss to enforce consistent history usage, achieving efficiency with minimal performance degradation. Extensive experiments on mainstream GUI navigation benchmarks demonstrate the strong performance of our model. Despite its smaller size, HiconAgent-3B outperforms GUI-R1-7B by **+8.46%** grounding and **+11.32%** step successful rate on GUI-Odyssey, while achieving comparable results on AndroidControl and AITW, with up to **2.47× computational speedup** and **60% FLOPs reduction**.

1. Introduction

Multimodal Large Language Model (MLLM)-based GUI agents [11, 46] have recently achieved strong performance in grounding and navigation tasks by leveraging textual instructions, visual observations, and historical trajectories. Among various training strategies, reinforcement learning (RL) [28, 33, 44] has demonstrated strong effectiveness by

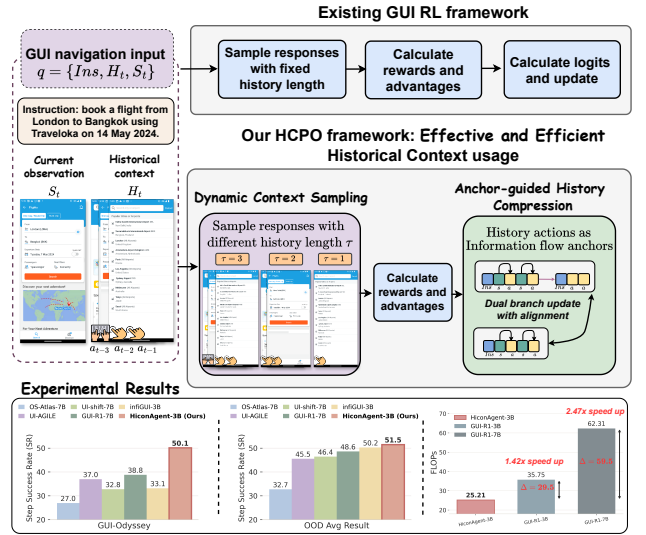


Figure 1. Comparison of existing GUI RL framework with our HCPO framework. HCPO jointly improves the sampling and update phases of training by integrating Dynamic Context Sampling (DCS) and Anchor-guided History Compression (AHC).

allowing GUI agents to directly optimize task-oriented objectives such as grounding accuracy and successful rate. Compared to supervised learning approaches [14, 49], RL-based methods [30, 31] significantly improve decision quality, robustness and generalization. As a result, RL has become a mainstream paradigm for training advanced GUI agents.

Despite these advances, the role of **history usage** in RL-based GUI agents remains largely underexplored. Most prior works [26, 30, 31] adopt a simplified design in which history observations (past screenshots) are omitted, and only history actions are included as the input context. While this choice reduces memory and computational cost, it discards rich visual cues from past observations that are often essential for resolving ambiguous instructions, grounding visually similar elements, and maintaining temporal consistency across steps [51]. Conversely, naively incorporating complete history, including both past actions and observa-

tions, substantially increases computational overhead due to the quadratic complexity of attention mechanisms and the large number of visual tokens from high-resolution screenshots. This trade-off between decision quality and efficiency motivates the development of methods that can effectively retain the most informative parts of historical context while mitigating redundancy.

To this end, we propose History Context-aware Policy Optimization (**HCPO**), a training framework designed to improve both the effectiveness and efficiency of history usage in GUI agents. As illustrated in Figure 1, HCPO improves both the sampling and update phases of existing GUI RL framework through two complementary components: Dynamic Context Sampling (**DCS**) and Anchor-guided History Compression (**AHC**).

In the sampling phase, DCS addresses the variability of history dependence across decision steps. Different from conventional RL approaches that use a fixed-length history, DCS samples multiple history variants for each rollout using the exponential-biased distribution, encouraging the model to adaptively utilize the most relevant context. Our sampling distribution is motivated by empirical observations, ensuring stable learning and avoiding the degeneration observed with naive uniform sampling. *In the update phase*, AHC jointly optimizes compressed and uncompressed branches to enhance policy updates under history compression. The compressed branch drops history observations after early fusion, retaining only action tokens as anchors to preserve essential decision signals. It is jointly optimized with the uncompressed branch through a history-enhanced alignment loss. This compression strategy is guided by our empirical layer-wise token-drop analysis. **Our contributions** can be summarized as follows:

- We conduct a **comprehensive empirical analysis** on history usage in GUI agents. Our findings show that different tasks and decision steps prefer different history lengths and history actions act as critical anchors for visual information flow. These findings reveal important inefficiencies in existing designs and directly motivate our method.
- We propose History Context-aware Policy Optimization (**HCPO**), a novel reinforcement fine-tuning framework that combines Dynamic Context Sampling and Anchor-guided History Compression. Together, they enable agents to learn adaptive history usage while reducing redundancy and preserving decision quality.
- We validate our method on three GUI navigation benchmarks: GUI-Odyssey, AndroidControl, and AITW. HiconAgent-3B consistently outperforms existing reinforcement learning based agents. It outperforms larger 7B agents such as GUI-R1-7B by **+11.32%** in step successful rate on GUI-Odyssey, while achieving **2.47×** computational speedup and a **60%** reduction in FLOPs.

2. Related Work

2.1. GUI Agents and History Utilization

GUI agents aim to complete high-level tasks by interacting with graphical user interfaces through sequences of low-level actions such as clicking and typing. Recent advances in multimodal large language models [1, 3–5, 15, 16, 19–23, 25, 34, 38–41, 43, 53, 55, 56] have rapidly expanded their capability to integrate and reason over diverse input modalities. This progress has enabled GUI agents to achieve significant improvements in multi-step navigation tasks [6, 8, 10, 11, 24, 35, 45–48].

Although historical information, including past observations and actions, is essential for GUI agents to understand user instructions and make decisions, the issue of redundancy in long histories remains a persistent challenge. Existing studies [7, 29, 52] under supervised fine-tuning show that adding past actions improves performance with minimal input cost. In contrast, incorporating full visual history brings larger gains but incurs significantly higher computation, especially in long-horizon or high-resolution scenarios. This trade-off highlights the inefficiency of naively using full history and motivates more selective or compressed representations of historical context.

2.2. Rule-based Reinforcement Learning

Large language models have shown remarkable improvements in reasoning-intensive tasks such as mathematics and programming when optimized with reinforcement learning. This reinforcement fine-tuning paradigm is now being actively explored in the multimodal domain. In particular, Group Relative Policy Optimization (GRPO) [12, 42] has emerged as an effective alternative to conventional methods such as PPO [37], especially for training multimodal models. It simplifies the training pipeline by evaluating responses using relative, normalized rewards computed within each sample group.

In the GUI domain, rule-based RL typically uses exact match or distance-based criteria for action types, coordinates, or textual inputs, providing fine-grained rewards that guide the agent toward correct execution [31, 32]. Although existing RL approaches have achieved notable performance gains, there has been little to no discussion on *effective and efficient history utilization*, and existing methods typically overlook strategies for balancing the trade-off between computational cost and the retention of critical historical context. Our method extends rule-based RL with *history-aware policy optimization* to improve decision quality while reducing computational overhead.

3. Preliminaries

Problem Definition. The goal of a GUI agent is to complete high-level tasks by interacting with graphical user interfaces

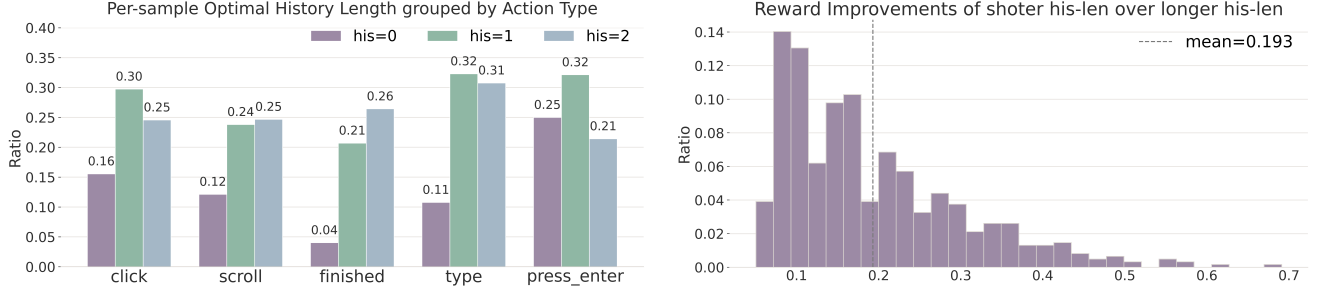


Figure 2. **Different samples prefer different history lengths.** *Left:* For each sample we evaluate a set of different history lengths τ and take the τ that yields the highest mean reward. The preferred τ differs across samples and action types. *Right:* Providing more history does not necessarily yield the optimal result, suggesting effective usage of historical information is under exploration.

through a sequence of low-level actions. At each timestep t , the agent observes a natural language task instruction I , a current screenshot observation s_t , and a history context $H_t = (s_{t-\tau}, a_{t-\tau}), \dots, (s_{t-1}, a_{t-1})$, where τ denotes the history window size, which is the number of previous steps included in the history. The agent then generates an action a_t conditioned on (I, H_t, s_t) and executes it in the GUI environment. This interaction process defines a sequential decision-making problem, which can be formulated as a Markov Decision Process (MDP).

Application of GRPO in GUI Agents. When applying GRPO to training GUI agents, we treat the agent as a policy model and reorganize the inputs by defining $q = (I, H_t, s_t)$. For each input q , the policy model generates a set of candidate responses $O = o_1, \dots, o_G$, where each response o_i consists of a thought process t_i and a corresponding action a_i , i.e., $o_i = (t_i, a_i)$. The standard GRPO algorithm adopts the following training objective:

$$\mathcal{L}_{GRPO} = -\mathbb{E} \left[\sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \text{KL}(\pi_{\theta}(o_i | q) \parallel \pi_{\theta_{ref}}(o_i | q)) \right) \right] \quad (1)$$

where ϵ is the clipping parameter, β is a hyperparameter. A_i is the advantage calculated by normalizing the group level rewards $\{R_i\}_{i=1}^G$. By normalization, advantage A_i represents the relative quality of the i -th response.

4. Rethinking History Usage: Limitations of Fixed Context and the Anchoring Role of Actions

A key challenge in building strong GUI agents is how to appropriately leverage historical context. To address this, we conduct two empirical studies focusing on effectiveness and efficiency. The first study analyzes how different samples benefit from different history lengths, showing that a fixed-length context is often suboptimal. This highlights the need for dynamically adjusting context length to match

step-specific dependencies. The second study examines how historical information flows through the model and finds that action tokens serve as anchors for aggregating and delivering useful visual semantics. This highlights the importance of retaining history actions while compressing redundant visuals to improve efficiency without compromising performance.

4.1. Analyzing the Impact of History Length

In GUI navigation, different steps may depend on varying lengths of history context. We conduct an empirical analysis to measure the impact of history length on decision quality across different samples and action types. As shown in the *left* plot of Figure 2, we perform rollout-based evaluation over the training set using a base model with fixed policy weights. For each sample, we conduct 8 rollouts under different history lengths $\tau \in \{0, 1, 2\}$, and record the average reward under each setting. We then determine the optimal history length per sample by comparing these mean rewards. Samples with negligible differences (mean reward difference < 0.05) are discarded. The result reveals a clear pattern: different samples exhibit different optimal history lengths. While some steps benefit from short-term context, others require longer to achieve higher reward.

Notably, we further analyze cases where shorter history lengths outperform longer ones, as shown in the *right* plot of Figure 2. For each such sample, we compute the mean-reward difference between the better-performing shorter context and a longer one, i.e., $\text{Improvement} = \text{mean_reward}(\tau_{\text{short}}) - \text{mean_reward}(\tau_{\text{long}})$. The resulting distribution illustrates that these improvements are non-trivial, shorter histories can yield significantly higher rewards in certain situations. This phenomenon suggests that longer history is not always more beneficial, and in some cases may even hinder performance (lower reward), likely due to introducing irrelevant information. Together with the left plot, this reinforces the insight that a fixed context length τ cannot accommodate the diverse temporal dependencies across samples. Dynamically varying the length of historical context can lead to more effective model behavior and improved decision quality.

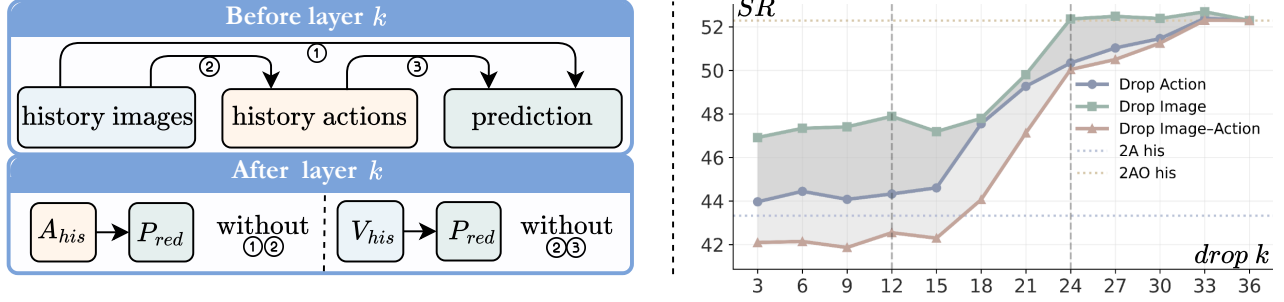


Figure 3. **Layer-wise token-drop analysis.** *Left:* Schematic of the layer-wise token-drop probe, illustrating the information flow of image-drop and action-drop. *Right:* Dropping A_{his} at shallow depths ($k < 12$) causes a much larger decline than dropping V_{his} . Even if rich visual information is retained, later layers cannot directly extract effective cues from V_{his} without the action anchors. As k increases, the action-drop curve rises toward the image-drop curve and the image-action drop curve converges rapidly.

4.2. History Actions as Information Flow Anchors

Naively appending historical observations to the context inflates sequence length and computation cost, burdening the model with tokens that contribute little to decision making [50]. In practice, only the regions associated with past actions tend to carry meaningful semantics for decision-making, while most other areas tend to be redundant. This motivates designing a method to compress redundant historical information to improve efficiency without harming performance. The central question is which tokens should be preserved during layer interaction. Different from previous work [7, 9] that adopts conclusions from information flow studies of single-image VQA scenario [54] to identify tokens for retention, we conduct an information flow analysis in the GUI navigation scenario within a reinforcement learning framework, tracing how historical visuals and actions interact across model depth.

Layer-wise token-drop setup. We probe how history propagates through the LLM with a layer-conditioned token drop. Qwen2.5-VL-3B (36 layers) is trained on our dataset (Section 6) with history image/action context and evaluated on AndroidControl using step successful rate (SR). At depth k , we remove from layer $k+1$ onward: (i) history actions $A_{his} = \{a_{t-\tau}, \dots, a_{t-1}\}$; (ii) history images $V_{his} = \{s_{t-\tau}, \dots, s_{t-1}\}$; or (iii) both history actions and images $H_t = (s_{t-\tau}, a_{t-\tau}), \dots, (s_{t-1}, a_{t-1})$. Sweeping k yields the curves in Figure 3; gaps between them show that later layers access history chiefly via action tokens, with a smaller direct contribution from visuals.

Shallow depth ($k < 12$): As shown in Figure 3, dropping A_{his} in shallow layers causes a large performance degradation, while dropping V_{his} at the same depths is much less harmful. This pattern indicates that effective use of historical information *depend on the action tokens as the anchor*: even if rich visual information is retained, later layers cannot directly extract effective cues from V_{his} without the action anchors. By contrast, dropping V_{his} in this range is more tolerable, since the model has already merged part of the

visual history into A_{his} ; the action anchors then pass that information forward through subsequent layers.

Deeper depth ($k \geq 12$): For $k > 12$, the *action-drop* curve rises quickly and, by mid depth, its gap to the *image-drop* curve is small. This indicates that, in these layers, predictions obtain historical information through interactions with the action anchors. Once that interaction has already occurred before the drop point, removing A_{his} or V_{his} later causes only little decline, and performance continues to improve with depth. When $k \geq 24$, all three curves converge to the no-compression accuracy, indicating that the model has already integrated most historical information and can proceed without retaining these history tokens.

Key discovery. From the layer-wise drop analysis in Figure 3, relying only on history actions (2A) yields limited benefit: without their paired screenshots, history action tokens A_{his} lack grounded semantics and provide little guidance for decision-making. In contrast, incorporating history observations (2AO) proves critical for navigation success. However, the gain does not arise from directly attending to history images. Instead, it emerges mainly at intermediate depths, where A_{his} interacts with the history visual tokens V_{his} and delivers the extracted information to subsequent tokens, highlighting the role of history actions as anchors for multimodal information flow.

Implications for designing compression methods. Guided by the above experimental results and analysis, we adopt a single rule to balance efficiency and effectiveness: *compress history only after an early fusion depth k , pruning V_{his} while retaining A_{his} .* On the *efficiency* side, removing history images greatly reduces sequence length and computation cost; on the *effectiveness* side, keeping the action anchors preserves the historical cues that later tokens actually use. To make the proposed compression method work better, training is essential to strengthen cross-modal interaction in the first k layers, so that by the time compression occurs, the relevant visual context has been sufficiently transferred into A_{his} , thereby preserving the necessary information flow.

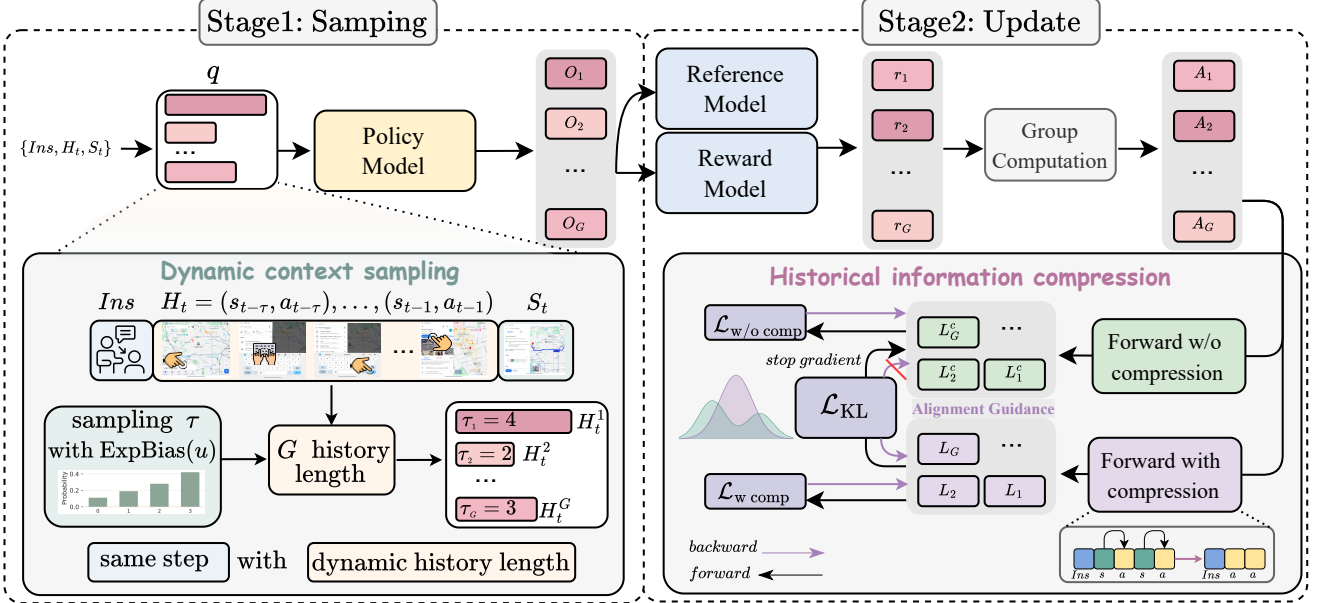


Figure 4. Overview of our history context-aware optimization framework for building HiconAgent. HCPO improves both the sampling and update phases of policy optimization by incorporating two key components: (1) **Dynamic Context Sampling (DCS)**, which introduces varied history lengths during training to encourage context-effective decision-making, and (2) **Anchor-guided History Compression (AHC)**, which adopts a dual-branch architecture where both branches share sampled responses and group-wise advantages. The compressed branch is trained using policy gradients, aligned with the uncompressed branch via a history-enhanced alignment loss.

5. HiconAgent

We propose History Context-aware Policy Optimization (HCPO), a reinforcement learning framework that improves history utilization by strengthening both the sampling and update phases. Dynamic Context Sampling (DCS) varies history lengths during rollouts to guide effective context usage, while Anchor-guided History Compression (AHC) reduces redundancy during updates via alignment with full-history supervision. Together, they enable policy optimization with effective and efficient historical context usage.

5.1. Dynamic Context Sampling

As shown in the analysis in Section 4, relying on a fixed-length history context is often suboptimal. This motivates adapting the history length to step-specific dependencies, allowing the policy to leverage the appropriate amount of context as needed. To address this, we propose Dynamic Context Sampling, which exposes the policy to diverse history lengths during training. This allows the agent to adaptively focus on the most relevant context, thereby improving overall policy learning.

During training, instead of always feeding a fixed-length history H_t , we dynamically sample and select G variants of truncated histories $\{H_t^1, \dots, H_t^G\}$, where H_t^i uses a selected history length $\tau_i \leq \tau$ sampled from distribution p . However, naively adopting a uniform distribution leads to a degeneration phenomenon, as later shown in our sampling ablation. Motivated by this empirical observation, we pro-

pose an exponential-biased distribution that mitigates training collapse by gradually shifting the sampling toward larger τ as training progresses. At training step u , our exponential-biased distribution $\text{ExpBias}(u)$ is defined as:

$$P(\tau_i | u) = \frac{\exp(\lambda(u) \tau_i)}{\sum_{j=0}^N \exp(\lambda(u) j)} \quad (2)$$

where $\lambda(u)$ is a linear function that increases with u . In the early stage of training, $\lambda(u) \approx 0$ and the distribution is nearly uniform, encouraging random exploration. As training progresses, $\lambda(u)$ gradually grows, yielding an increasingly biased exponential distribution that favors larger values of τ_i . This schedule smoothly shifts the sampling strategy from random selection to full-context history.

Each variant forms an input $q_i = (I, H_t^i, s_t)$, and produces a response o_i . These G responses are evaluated as a group, yielding an advantage value for each response. Responses with higher advantages receive stronger gradient updates, allowing the policy to adaptively learn which history lengths lead to improved policy behavior. Importantly, to maintain consistency between training and inference, we combine each sampled response o_i with the full history context input (I, H_t, s_t) to compute the logits for optimization. This design allows the policy to explore effective solutions under varying historical conditions while being evaluated under a unified context length.

5.2. Anchor-guided History Compression

Leveraging the observation in Section 4 that history actions preserve the historical cues that later tokens actually use, we propose Anchor-Guided History Compression. AHC keeps past actions as anchors, prunes visual history after early fusion, and leverages alignment with an uncompressed branch, preserving decision quality while reducing sequence length and FLOPs.

Let $q = \{I, H_t, s_t\}$ be the model input at step t , and define the importance sampling ratio as $\rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}$. The responses from the uncompressed branch are optimized using the standard GRPO objective:

$$\mathcal{L}_{\text{w/o comp}} = - \sum_{i=1}^G \left(\min(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i) - \beta \text{KL}[\pi_\theta(o_i | q) \parallel \pi_{\theta_{\text{ref}}}(o_i | q)] \right), \quad (3)$$

To reduce redundancy, we remove all history vision tokens $V_{\text{his}} = \{s_{t-\tau}, \dots, s_{t-1}\}$ while retaining the past action tokens. The model continues forwarding with the compressed sequence $\{I, A_{\text{his}}, s_t\}$, where the retained action tokens $A_{\text{his}} = \{a_{t-\tau}, \dots, a_{t-1}\}$ constitute the compressed history H_t^c . The compressed branch is further optimized with a GRPO-style objective using the same responses and advantages, leveraging the superior quality of responses generated from the uncompressed branch, where $q^c = \{I, H_t^c, s_t\}$ denotes the inputs with compressed history and the importance sampling ratio as $\rho_i^c = \frac{\pi_\theta(o_i|q^c)}{\pi_{\theta_{\text{old}}}(o_i|q^c)}$.

$$\mathcal{L}_{\text{w/ comp}} = - \sum_{i=1}^G \left(\min(\rho_i^c A_i, \text{clip}(\rho_i^c, 1 - \epsilon, 1 + \epsilon) A_i) - \beta \text{KL}[\pi_\theta(o_i | q^c) \parallel \pi_{\theta_{\text{ref}}}(o_i | q^c)] \right). \quad (4)$$

To ensure that the compressed branch retains the core decision-making ability of the original model, we introduce a *history-enhanced alignment objective*. Specifically, given the original history H_t and its compressed counterpart H_t^c , we perform parallel forward passes through both branches. We then minimize the KL divergence between their output distributions, effectively using the uncompressed branch as a teacher to guide the compressed branch. Note that the uncompressed branch is used only for guidance; we detach its outputs to prevent gradient backpropagation in the KL loss. This approach allows the compression module to reduce redundancy while preserving critical behavioral patterns from the original branch. The alignment objective is defined as:

$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^G \text{KL}[\pi_\theta(o_i | q^c) \parallel \pi_\theta(o_i | q)], \quad (5)$$

The final HCPO loss is the sum of the uncompressed branch loss, the compressed branch loss, and the alignment con-

straint:

$$\mathcal{L}_{\text{HCPO}} = \mathcal{L}_{\text{w/o comp}} + \mathcal{L}_{\text{w comp}} + \lambda \mathcal{L}_{\text{KL}}, \quad (6)$$

where λ controls the strength of the alignment guidance. This framework enables effective policy optimization under compressed context while preserving temporal consistency.

5.3. Reward Design

In GUI navigation tasks, each action consists of a type and a value. The type is chosen from a set of discrete options (e.g., CLICK, SCROLL), while the value format varies depending on the type. Based on this characteristic of GUI tasks, we introduce the following three rewards:

Format reward (r^f): This term ensures the model’s response adheres to a predefined structure like `<think>...</think><answer>...</answer>`. It returns 1 if the format is correct, 0 otherwise, promoting structured reasoning and output generation.

Action type reward (r^t): We assign 1 if the predicted action type exactly matches the ground-truth type, and 0 otherwise. This term enforces correctness at the semantic level of action selection.

Action value reward (r^v): For actions *without* values (e.g., PRESS.BACK), the reward is 1 if the type is correct. For actions with *textual* values (e.g., TYPE, OPEN_APP), we compute the F1 score between prediction and ground truth, awarding 1 if $F1 > 0.5$. For actions with *discrete* values (e.g., SCROLL), the value must exactly match the ground truth. For *coordinate-based* actions (e.g., CLICK), we calculate the Euclidean distance d between predicted and actual coordinates, and assign a continuous reward $r^v = 1 - d$ to allow fine-grained feedback for grounding accuracy.

The final reward used for policy optimization is the sum of the three components:

$$r = r^f + r^t + r^v \quad (7)$$

6. Experiments

6.1. Implementation Details

Metrics. We evaluate our model on three representative navigation-oriented datasets, AndroidControl-High[18], AITW[17] and GUI-Odyssey[29], considering only their test splits under an out-of-distribution (OOD) evaluation setting to assess generalization performance. We use three standard metrics widely adopted in prior work on GUI agents, following the evaluation protocol of Os-Atlas [46]. Specifically, we report action type prediction accuracy (**Type**), GUI grounding accuracy (**Grounding**), and step success rate (**SR**).

FLOPs is computed using deepspeed flops-profiler with batch size 1. We include all model components when calculating FLOPs. The reported number is averaged over 200 samples from the training set with history length $\tau = 2$.

Training and Evaluation. HiconAgent-3B is built upon Qwen2.5-VL-3B. We follow the same training setting in GUI-R1 [31]. In the reinforcement fine-tuning stage, we adopt the following hyperparameter settings to ensure stable optimization. The rollout batch size and global batch size is set to 64, with 8 rollouts per update step. We use a small learning rate of 1×10^{-6} to stabilize training. To balance computational efficiency and generation quality, the maximum number of input pixels is capped at 1,003,520. For the language input and output, both the maximum prompt length and the maximum response length are restricted to 2048 tokens. The rollout temperature is fixed at 1.0 to encourage diverse yet consistent exploration during training.

To keep the number of tokens consistent with prior work, we set the history window size to 2, meaning the agent can access up to two past interaction steps. Each historical step includes both the screenshot observation and the corresponding action, i.e., $H_t = \{(s_{t-2}, a_{t-2}), (s_{t-1}, a_{t-1})\}$. All images are resized to a fixed resolution, and the number of visual tokens after encoding is limited to a maximum of 512 to ensure computational efficiency.

Model Configuration. From an efficiency perspective, HiconAgent-3B adopts the drop $k = 6$ configuration by default in the following experiments. This setting achieves up to a 60% reduction in FLOPs while maintaining competitive accuracy, as shown in Table 5.

6.2. Experiment Results

We present the main experimental results in Table 1 and Table 2 on three representative GUI navigation datasets: AndroidControl-High [18], AITW [36] and GUI-Odyssey [29]. Table 1 provides a detailed comparison under the same data scale and training settings, highlighting the effect of our history-aware optimization strategy against both supervised fine-tuning and reinforcement fine-tuning baselines. Table 2 further extends the comparison to recent advanced GUI agents of varying model sizes and training data volumes, demonstrating the generalization ability of our approach in out-of-distribution (OOD) scenarios.

As shown in Table 1, HiconAgent-3B achieves consistent improvements over all baselines trained under the same data scale and training settings. We first observe that HiconAgent-3B consistently outperforms the supervised baseline Qwen2.5VL-3B, with clear gains in both grounding accuracy and step successful rate. When trained with reinforcement learning, HiconAgent-3B further surpasses the GRPO baseline GUI-R1-3B (+5.85% SR) and GUI-R1-7B (+0.73% SR) on AndroidControl under the same training setup, demonstrating a stronger decision-making ability.

The advantage of HiconAgent becomes especially pronounced on the long-horizon GUI-Odyssey benchmark. Despite having less than half the parameters of GUI-R1-7B, our 3B model achieves a remarkable +8.46% improvement

Models	AC-High		GUI-Odyssey	
	Grounding	SR	Grounding	SR
Supervised Fine-Tuning				
GUI-R1-3B [31]	49.53	41.22	32.21	27.31
GUI-R1-7B [31]	58.69	48.11	38.65	34.44
Qwen2.5VL-3B	52.89	41.95	36.21	31.74
Zero-Shot				
GPT-4o [15]	30.90	21.17	14.17	5.36
Qwen2.5VL-3B	46.51	38.90	26.49	26.69
Qwen2.5VL-7B	59.71	47.06	37.78	34.37
Reinforcement Fine-Tuning				
GUI-R1-3B [31]	56.24	46.55	41.52	41.33
GUI-R1-7B [31]	65.56	51.67	43.64	38.79
HiconAgent-3B	65.51 (-0.05)	52.40 (+0.73)	52.10 (+8.46)	50.11 (+11.32)

Table 1. Performances on **AndroidControl-High** and **GUI-Odyssey**. Red indicates improvement, green indicates degradation compared to GUI-R1-7B. Our 3B model outperforms GUI-R1-7B by **+8.46%** grounding and **+11.32%** SR on GUI-Odyssey.

in grounding accuracy and +11.32% in step successful rate. Compared with GUI-R1, which does not explicitly exploit historical information, HiconAgent-3B adopts a more effective strategy for leveraging historical context, leading to stronger sequential reasoning and consistent execution.

Table 2 summarizes OOD generalization across the three benchmarks. Without relying on large-scale data filtering or dataset curation, HiconAgent is trained on only 3K unfiltered samples, yet it achieves the highest average step successful rate (51.47%) among all compared models. Remarkably, it surpasses stronger models such as OS-Atlas-7B, GUI-R1-7B, and infiGUI-3B, which are trained with much larger data volumes (13M-filtered, 3K-filtered, and 32K-filtered respectively). Although infiGUI-3B achieves a notably high SR on the AndroidControl benchmark (71.1%), its performance drops considerably on AITW and GUI-Odyssey, indicating weaker generalization under OOD conditions. This demon-

Model	#Data Filter	AC-High	AITW	Odyssey	Avg SR
GPT-4o [15]	— —	21.17	26.07	5.36	17.54
OS-Atlas-7B [46]	13M ✓	29.83	41.38	26.96	32.72
GUI-R1-7B [31]	3K ✓	51.67	55.31	38.79	48.59
UI-shift-7B [13]	2K ✓	52.16	54.38	32.75	46.43
UI-AGILE [26]	9K ✓	50.97	48.66	36.96	45.53
infiGUI-3B [27]	32K ✓	71.10	46.51	33.15	50.25
HiconAgent-3B	3K ✗	52.40	51.91	50.11	51.47

Table 2. Step Successful Rate evaluated on three representative GUI navigation datasets, compared with recent models. Cells highlighted in red correspond to datasets that are IID for the respective models, whereas cells highlighted in green indicate OOD setting.

Models	DCS	Update τ	Sampling $p(\tau)$	AC-High SR	Train Hrs
HCPO (w/o DCS)	–	2	–	51.03	17h
HCPO (Uniform)	✓	2	$U(0, 2)$	50.53	17h
HCPO (Uniform)	✓	$\{0, 1, 2\}$	$U(0, 2)$	51.62	30h
HCPO (ExpBias)	✓	2	$\text{ExpBias}(u)$	52.40	17h

Table 3. Ablation study on different sampling distributions in Dynamic Context Sampling (DCS). $U(0, 2)$ denotes uniform distribution over $\tau \in \{0, 1, 2\}$, while $\text{ExpBias}(u)$ represents the exponential-biased schedule defined in Eq. (2).

strates that our HCPO framework is not only effective but also highly data-efficient, enabling strong generalization even without large-scale data curation.

6.3. Ablation Study

Impact of sampling distribution p . We perform an ablation on the choice of $p(\tau)$ to study how different sampling strategies affect learning. A uniform sampling baseline $U(0, 2)$ is first adopted for comparison, where $\tau \in \{0, 1, 2\}$ denotes the sampled history length. Under plain uniform sampling $U(0, 2)$, we observe a degeneration phenomenon during training: the sampling quality of the shorter histories ($\tau = 0, 1$) weakens over time as shown in Figure 5. This is because we only compute gradients and update parameters using the context with $\tau = 2$ to ensure training–inference consistency. To mitigate this, we design a variant that enforces the inclusion of all input contexts with $\tau \in \{0, 1, 2\}$ in the optimization. While this all- τ update strategy partially recovers the performance lost by naive uniform sampling, it substantially increases training overhead and still fails to match the performance of our proposed exponential-biased distribution. The exponential-biased sampling $\text{ExpBias}(u)$ mitigates collapse by gradually biasing the sampling toward larger τ as training progresses, while still encouraging exploration of shorter histories in the early stage. This strategy achieves the best trade-off between performance and computational cost, as reported in Table 3.

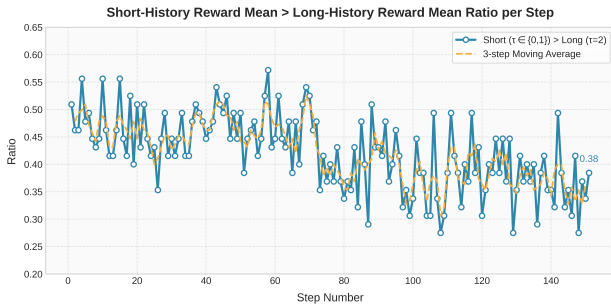


Figure 5. Evolution of the short-vs-long history reward ratio under uniform τ sampling. The declining ratio reflects the gradual degradation of short-history response quality during training.

Effect of DCS and AHC. Table 4 evaluates the impact of Dynamic Context Sampling (DCS) and Anchor-guided

History Compression (AHC) across the three navigation benchmarks. Training only a compressed branch using standard GRPO yields the weakest performance, indicating that unguided compression fails to retain useful historical signals. Adding an uncompressed branch without KL improves performance, showing that full-history responses help guide the compressed branch. Incorporating KL alignment between the two branches further enhances performance, validating the benefit of matching compressed outputs to the full-history teacher. Finally, enabling DCS achieves the best overall results, confirming that DCS enhances the model’s ability to utilize historical context more effectively.

Models	2Branch	KL	DCS	AC-High	AITW	Odyssey
GRPO	–	–	–	44.89	45.62	43.21
HCPO (w/o KL, DCS)	✓	–	–	48.70(+3.81)	49.23(+3.61)	47.09(+3.88)
HCPO (w/o DCS)	✓	✓	–	51.03(+6.14)	50.78(+5.16)	48.68(+5.47)
HCPO	✓	✓	✓	52.40(+7.51)	51.91(+6.29)	50.11(+6.90)

Table 4. Ablation study on the dual-branch architecture, alignment loss, and DCS, evaluated using the SR metric. Experiments are conducted with compression enabled.

Impact of layer drop position. We study how the position of the layer drop (k) affects the trade-off between computation and performance. As shown in Table 5, dropping earlier layers (e.g., $k = 1$) leads to the largest FLOPs reduction (−33.81 % vs 3B), but comes at the cost of degraded performance across all metrics. As k increases, computation gradually increases while performance steadily improves. Our choice of $k = 6$ offers a good balance, retaining competitive performance with a 59.54% FLOPs reduction and 2.47× computational speedup (62.31T / 35.75).

Drop k	Computation				Performance	
	FLOPs	Δ vs 3B	Δ vs 7B	Tokens	Type	SR
$k = 1$	23.66	−33.81%	−62.02%	674	65.33	46.11
$k = 3$	24.28	−32.07%	−61.04%	674	66.00	46.92
$k = 6$	25.21	−29.47%	−59.54%	674	66.56	47.34
$k = 12$	27.07	−24.28%	−56.57%	674	67.02	47.89
w/o drop	35.75	–	−42.64%	1664	69.29	52.29

Table 5. FLOPs, token counts, and performance of base model under different layer-drop settings, with relative FLOPs reduction versus the 3B model (35.75T) and 7B model (62.31T).

Impact of HCPO on History Utilization across Action Types. Beyond the overall improvements reported in Table 4, we further examine per-action performance to understand how HCPO influences different types of decisions. As shown in Figure 7, applying HCPO consistently improves accuracy across most action categories on both AndroidControl and GUI-Odyssey. The gains are especially pronounced on finished actions, which directly determine whether a trajectory

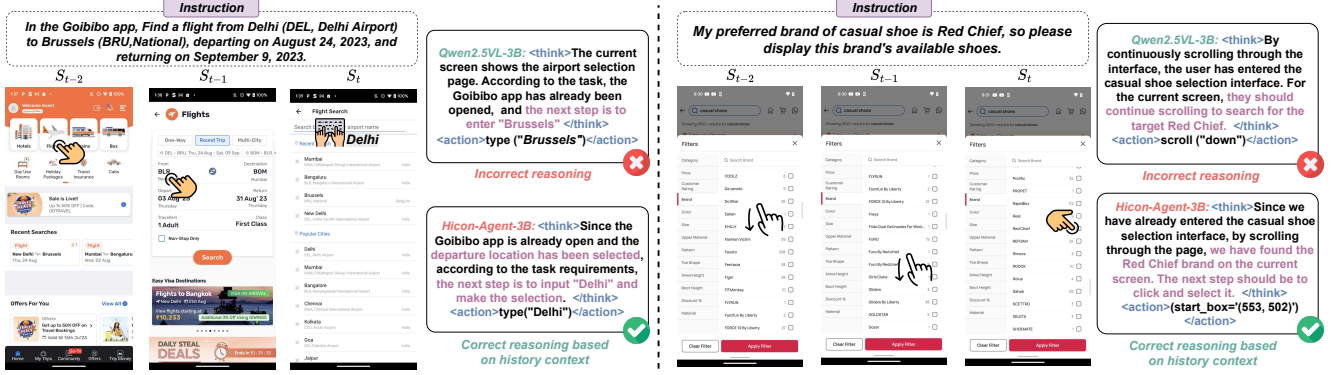


Figure 6. To illustrate HCPO’s enhancement in leveraging historical information, we present two scenarios: Left (Flight Booking) and Right (Shopping Task). Our model correctly inputs *Delhi* by reasoning over historical context and selects *Red Chief* despite visual redundancy. While the base model trained without HCPO misinterprets history and fails in both cases.

is successfully terminated, suggesting that HCPO strengthens the model’s ability to maintain global sequence control rather than overfitting to local step transitions. Improvements are also evident in challenging categories such as scroll, which demands exact direction prediction, and type, which often involves generating context-relevant input. Such findings highlight that our history-aware training paradigm remains effective in scenarios where historical cues are essential for accurate action generation.

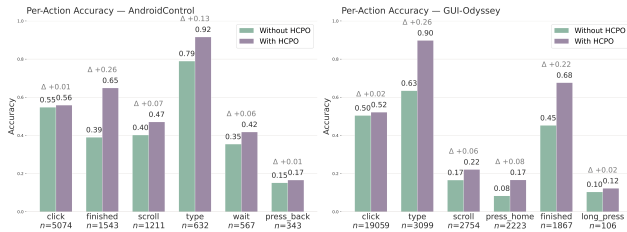


Figure 7. Per-action accuracy comparison before and after applying HCPO. Both AndroidControl (left) and GUI-Odyssey (right) benefit from history-compressed optimization, especially on *finished* actions, showing improved sequential decision quality.

Case Study on Historical Context Utilization. To investigate how HCPO intuitively enhances the use of historical context, we conduct a comparative case study on two representative tasks, as shown in Figure 6. In the flight-booking scenario (left), HiconAgent-3B correctly reasons over previous steps and inputs *Delhi* as the departure city, whereas the base model trained without HCPO mistakenly enters the destination *Brussels*. Similarly, in the shopping task (right), where repetitive history screenshots introduce visual ambiguity, our model robustly focuses on the current frame to accurately select the target brand *Red Chief*, while the base model fails to resolve the ambiguity and deviates from the intended action. These cases illustrate that HCPO enables more reliable and effective usage of historical context.

7. Conclusion

In this paper, we present HiconAgent, a history-aware GUI agent trained with History Context-aware Policy Optimization. Through extensive empirical investigations, we first revisited how history is utilized in GUI reinforcement learning agents. Our two key studies revealed that different decision steps prefer different history lengths and historical actions serve as information flow anchors. By pairing DCS and AHC, our model outperforms larger models with fewer FLOPs. These results highlight HiconAgent as a practical path toward lightweight, high-performance GUI agents.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 2
- [2] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Peng Gao, Shuai Ren, and Hongsheng Li. AMEX: Android Multi-annotation Expo Dataset for Mobile GUI Agents. *arXiv preprint*, 2024. arXiv:2407.17490 [cs]. 1
- [3] Gongwei Chen, Xinhang Song, Haitao Zeng, and Shuqiang Jiang. Scene recognition with prototype-agnostic scene layout. *IEEE Transactions on Image Processing*, 29:5877–5888, 2020. 2
- [4] Gongwei Chen, Xinhang Song, Bohan Wang, and Shuqiang Jiang. See more for scene: Pairwise consistency learning for scene classification. In *Advances in Neural Information Processing Systems*, pages 4987–4999. Curran Associates, Inc., 2021.
- [5] Gongwei Chen, Leyang Shen, Rui Shao, Xiang Deng, and Liqiang Nie. Lion: Empowering multimodal large language model with dual-level visual knowledge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26540–26550, 2024. 2
- [6] Gongwei Chen, Lirong Jie, Lexiao Zou, Weili Guan, Miao Zhang, and Liqiang Nie. Enhancing GUI agent with uncertainty-aware self-trained evaluator. In *The Thirty-ninth*

Annual Conference on Neural Information Processing Systems, 2025. 2

- [7] Gongwei Chen, Xurui Zhou, Rui Shao, Yibo Lyu, Kaiwen Zhou, Shuai Wang, Wentao Li, Yinchuan Li, Zhonggang Qi, and Liqiang Nie. Less is more: Empowering gui agent with context-aware simplification. *arXiv preprint arXiv:2507.03730*, 2025. 2, 4
- [8] Jingxuan Chen, Derek Yuen, Bin Xie, Yuhao Yang, Gongwei Chen, Zhihao Wu, Li Yixing, Xurui Zhou, Weiwen Liu, Shuai Wang, et al. Spa-bench: A comprehensive benchmark for smartphone agent evaluation. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024. 2
- [9] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024. 4
- [10] Xinyang Chen, Sinan Wang, Mingsheng Long, and Jianmin Wang. Transferability vs. discriminability: Batch spectral penalization for adversarial domain adaptation. In *International conference on machine learning*, pages 1081–1090. PMLR, 2019. 2
- [11] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. arXiv, 2024. arXiv:2401.10935 [cs]. 1, 2
- [12] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhan Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yao-hui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025. arXiv:2501.12948 [cs]. 2
- [13] Longxi Gao, Li Zhang, and Mengwei Xu. Uishift: Enhancing vlm-based gui agents through self-supervised reinforcement learning. *arXiv preprint arXiv:2505.12493*, 2025. 7
- [14] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. 1
- [15] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 2, 7
- [16] Junpeng Jiang, Gangyi Hong, Miao Zhang, Hengtong Hu, Kun Zhan, Rui Shao, and Liqiang Nie. Dive: Efficient multi-view driving scenes generation based on video diffusion transformer. *arXiv preprint arXiv:2504.19614*, 2025. 2
- [17] Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. AutoWebGLM: Bootstrap And Reinforce A Large Language Model-based Web Navigating Agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. arXiv, 2024. arXiv:2404.03648 [cs]. 6
- [18] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024. 6, 7
- [19] Wei Li, Bing Hu, Rui Shao, Leyang Shen, and Liqiang Nie. Lion-fs: Fast & slow video-language thinker as online video assistant. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 3240–3251, 2025. 2
- [20] Wei Li, Renshan Zhang, Rui Shao, Zhijian Fang, Kaiwen Zhou, Zhuotao Tian, and Liqiang Nie. Semanticvla: Semantic-aligned sparsification and enhancement for efficient robotic manipulation. *arXiv preprint arXiv:2511.10518*, 2025.
- [21] Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. *arXiv preprint arXiv:2508.21046*, 2025.
- [22] Zaijing Li, Gongwei Chen, Rui Shao, Yuquan Xie, Dongmei Jiang, and Liqiang Nie. Enhancing emotional generation

- capability of large language models via emotional chain-of-thought. *arXiv preprint arXiv:2401.06836*, 2024.
- [23] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. In *Advances in Neural Information Processing Systems*, pages 49881–49913, 2024. 2
- [24] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Weili Guan, Dongmei Jiang, and Liqiang Nie. Optimus-3: Towards generalist multimodal minecraft agents with scalable task experts. *arXiv preprint arXiv:2506.10357*, 2025. 2
- [25] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. Optimus-2: Multimodal minecraft agent with goal-observation-action conditioned policy. In *Proceedings of the computer vision and pattern recognition conference*, pages 9039–9049, 2025. 2
- [26] Shuquan Lian, Yuhang Wu, Jia Ma, Zihan Song, Bingqi Chen, Xiaowu Zheng, and Hui Li. Ui-agile: Advancing gui agents with effective reinforcement learning and precise inference-time grounding. *arXiv preprint arXiv:2507.22025*, 2025. 1, 7
- [27] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239*, 2025. 7
- [28] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-RFT: Visual Reinforcement Fine-Tuning, 2025. *arXiv:2503.01785 [cs]*. 1
- [29] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. GUI Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. *arXiv preprint*, 2024. *arXiv:2406.08451 [cs]*. 2, 6, 7
- [30] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Guanqing Xiong, and Hongsheng Li. UI-R1: Enhancing Action Prediction of GUI Agents by Reinforcement Learning, 2025. *arXiv:2503.21620 [cs]* version: 1. 1
- [31] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025. 1, 2, 7
- [32] Zhihao Luo, Wentao Yan and Jingyu Gong, Min Wang, Zhizhong Zhang, Xuhong Wang, Yuan Xie, and Xin Tan. Navimaster: Learning a unified policy for gui and embodied navigation tasks. *arXiv preprint arXiv:2508.02046*, 2025. 2
- [33] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. ReFT: Reasoning with Reinforced Fine-Tuning, 2024. *arXiv:2401.08967 [cs]*. 1
- [34] Yibo Lyu, Rui Shao, Gongwei Chen, Yijie Zhu, Weili Guan, and Liqiang Nie. Puma: Layer-pruned language model for efficient unified multimodal retrieval with modality-adaptive learning. In *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025. 2
- [35] Yibo Lyu, Gongwei Chen, Rui Shao, Weili Guan, and Liqiang Nie. Personalalign: Hierarchical implicit intent alignment for personalized gui agent with long-term user-centric records. *arXiv preprint arXiv:2601.09636*, 2026. 2
- [36] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023. 7
- [37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. pages 1–12, 2017. *arXiv: 1707.06347*. 2
- [38] Rui Shao, Xiangyuan Lan, and Pong C Yuen. Deep convolutional dynamic texture learning with adaptive channel-discriminability for 3d mask face anti-spoofing. In *2017 IEEE International Joint Conference on Biometrics (IJCB)*, pages 748–755. IEEE, 2017. 2
- [39] Rui Shao, Xiangyuan Lan, Jiawei Li, and Pong C Yuen. Multi-adversarial discriminative deep domain generalization for face presentation attack detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10023–10031, 2019.
- [40] Rui Shao, Tianxing Wu, and Ziwei Liu. Detecting and grounding multi-modal media manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6904–6913, 2023.
- [41] Rui Shao, Tianxing Wu, Jianlong Wu, Liqiang Nie, and Ziwei Liu. Detecting and grounding multi-modal media manipulation and beyond. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 2
- [42] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024. *arXiv:2402.03300 [cs]*. 2
- [43] Leyang Shen, Gongwei Chen, Rui Shao, Weili Guan, and Liqiang Nie. Mome: Mixture of multimodal experts for generalist multimodal large language models. In *Advances in Neural Information Processing Systems*, pages 42048–42070, 2024. 2
- [44] Huajie Tan, Yuheng Ji, Xiaoshuai Hao, Minglan Lin, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Reason-RFT: Reinforcement Fine-Tuning for Visual Reasoning, 2025. *arXiv:2503.20752 [cs]*. 1
- [45] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv preprint*, 2024. *arXiv:2401.16158 [cs]*. 2
- [46] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv preprint*, 2024. *arXiv:2410.23218 [cs]*. 1, 6, 7
- [47] Bin Xie, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Jie Liu, Min Zhang, and Liqiang Nie. Gui-explorer: Autonomous exploration and mining of transition-aware knowledge for gui agent. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [48] Yuquan Xie, Zaijing Li, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Dongmei Jiang, and Liqiang Nie. Mirage-1: Augmenting and updating gui agent with hierarchical multimodal skills. *arXiv preprint arXiv:2506.10387*, 2025. 2

- [49] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguviz: Unified Pure Vision Agents for Autonomous GUI Interaction. *arXiv preprint*, 2024. arXiv:2412.04454 [cs]. 1
- [50] Danyang Zhang, Situo Zhang, Ziyue Yang, Zichen Zhu, Zihan Zhao, Ruisheng Cao, Lu Chen, and Kai Yu. Progrm: Build better gui agents with progress rewards. *arXiv preprint arXiv:2505.18121*, 2025. 4
- [51] Jiwen Zhang, Yaqi Yu, Minghui Liao, Wentao Li, Jihao Wu, and Zhongyu Wei. Ui-hawk: Unleashing the screen stream understanding for gui agents. *Preprints*, 2024. 1
- [52] Jiwen Zhang, Yaqi Yu, Minghui Liao, Wentao Li, Jihao Wu, and Zhongyu Wei. UI-Hawk: Unleashing the Screen Stream Understanding for GUI Agents, 2024. 2
- [53] Renshan Zhang, Rui Shao, Gongwei Chen, Miao Zhang, Kaiwen Zhou, Weili Guan, and Liqiang Nie. Falcon: Resolving visual redundancy and fragmentation in high-resolution multimodal large language models via visual registers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. 2
- [54] Zhi Zhang, Srishti Yadav, Fengze Han, and Ekaterina Shutova. Cross-modal information flow in multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19781–19791, 2025. 4
- [55] Yijie Zhu, Rui Shao, Ziyang Liu, Jie He, Jizhihui Liu, Jiuru Wang, and Zitong Yu. H-gar: A hierarchical interaction framework via goal-driven observation-action refinement for robotic manipulation. *arXiv preprint arXiv:2511.17079*, 2025. 2
- [56] Lexiao Zou, Gongwei Chen, Yanda Chen, and Miao Zhang. Enhancing diffusion-based dataset distillation via adversary-guided curriculum sampling. In *2025 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6. IEEE, 2025. 2

A. Visualization of SSR curve during training

During training, we monitor the execution accuracy to evaluate the learning behavior of the policy over time. As shown in Figure 8, integrating DCS into the AHC framework leads to a clear improvement in training dynamics. The curve with DCS achieves both higher convergence speed and better final accuracy compared to the AHC-only baseline. This indicates that naively relying on fixed-length histories limits the model’s ability to generalize. In contrast, DCS adaptively determines the effective history length based on task complexity, enabling the agent to access relevant information while avoiding overfitting to noisy or irrelevant historical states. These results verify that dynamically sampled history promotes more stable optimization and enhances the agent’s capability to learn meaningful sequential dependencies.

B. Visualization of sampling distribution in DCS.

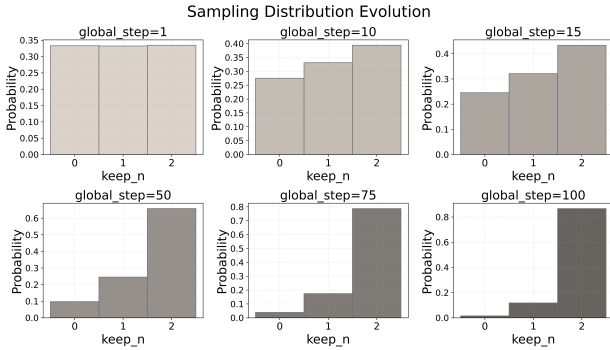


Figure 9. Evolution of the sampling distribution from uniform exploration to exponentially biased selection as training progresses.

We sample the number of preserved history blocks $\tau_i \in \{0, 1, 2\}$ from a time-dependent exponential distribution. At training step u , the sampling distribution is defined as

$$P(\tau_i | u) = \frac{\exp(\lambda(u) \tau_i)}{\sum_{j=0}^N \exp(\lambda(u) j)},$$

where $\lambda(u)$ is a linear function that increases with u . As shown in Figure 9, in the early stage of training, $\lambda(u) \approx 0$ and the distribution is nearly uniform, encouraging random exploration. As training progresses, $\lambda(u)$ gradually grows, yielding an increasingly biased exponential distribution that favors larger values of τ_i . This schedule smoothly shifts the sampling strategy from random selection to full-context history.

We define $\lambda(u)$ as a function that increases from 0 to $\lambda_{\max}$ within the first αT steps, gradually shifting the sampling distribution from uniform to exponentially biased:

$$\lambda(u) = \lambda_{\max} \cdot \min\left(1, \frac{u}{\alpha T}\right) \quad (8)$$

T denotes the total number of training steps. $\lambda_{\max}$ controls the final steepness of the exponential bias (larger $\lambda_{\max}$ yields a more strongly peaked distribution that favors larger τ_i), while α determines the fraction of training used for warm-up. In our implementation, we set $\lambda_{\max} = 2$, $\alpha = \frac{1}{3}$.

C. GUI datasets

We construct our training data from the open-source AMEX dataset[2], which contains high-level GUI interaction trajectories. Preserving the original action distribution, we randomly sample 3,000 steps without applying additional filtering or cleaning procedures.

AMEX AMEX is a large-scale mobile GUI dataset for training and evaluating control agents, comprising $>104K$ high-resolution screenshots from 110 Android apps with multi-level annotations. Each episode contains about 13 actions on average.

Action distribution (full dataset ratio):

- `click(start_box='(x, y)')` **24815 (64.11%)**
- `scroll(direction='down or up or right or left')` **7628 (19.71%)**
- `finished()` **2828 (7.31%)**
- `type(content='')` **2419 (6.25%)**
- `press_enter()` **651 (1.68%)**
- `impossible()` **220 (0.57%)**
- `press_back()` **135 (0.35%)**
- `press_home()` **13 (0.03%)**

AndroidControl AndroidControl is diverse benchmark to study data scaling for UI control, containing 15,283 demonstrations spanning 14,548 unique tasks across 833 Android apps, with both high-level and low-level human-written instructions for each task. Each episode contains about 5 actions on average.

Action distribution (test split):

- `click(start_box='(x, y)')` **5074 (50.81%)**
- `finished()` **1543 (15.45%)**
- `scroll(direction='down or up or right or left')` **1211 (12.13%)**
- `type(content='')` **632 (6.33%)**
- `open_app(app_name='')` **608 (6.09%)**
- `wait()` **567 (5.68%)**
- `press_back()` **343 (3.43%)**
- `long_press(start_box='(x, y)')` **9 (0.09%)**

GUI-Odyssey GUI-Odyssey is a cross-app mobile GUI navigation dataset for multi-step workflows across apps; the paper reports 7,735 episodes over 6 devices, 6 task types, 201 apps and $\sim 1.4K$ app combinations. Each episode contains about 15 actions on average.

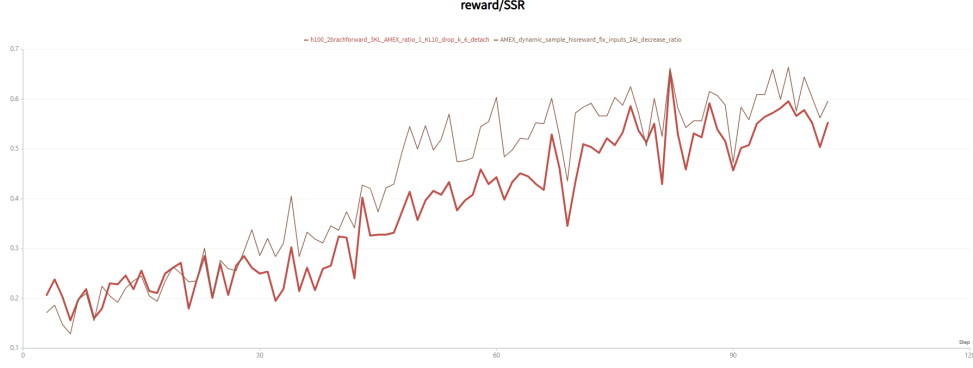


Figure 8. Training accuracy curves of Hicon-Agent with and without DCS under the AHC framework. Models trained with DCS exhibit consistently higher accuracy and faster convergence, demonstrating that adaptive history sampling facilitates more effective learning.

Action distribution (test split):

- `click(start_box='(x, y)')` **19142 (65.05%)**
- `type(content='')` **3113 (10.58%)**
- `scroll(direction='down or up or right or left')` **2764 (9.39%)**
- `press_home()` **2233 (7.59%)**
- `finished()` **1875 (6.37%)**
- `long_press(start_box='(x, y)')` **106 (0.36%)**
- `press_recent()` **74 (0.25%)**
- `press_back()` **61 (0.21%)**
- `impossible()` **58 (0.20%)**

AITW Android in the Wild (AITW) is a large-scale dataset for Android device control using natural language instructions. The paper reports 715,000 episodes, 30,000+ unique instructions, 8 device types (Pixel 2 XL through Pixel 6), 4 Android versions, covering hundreds of apps and websites. Each episode contains about 6.5 actions on average.

D. Effect of history observation and compression

We first examine the impact of historical observations by comparing Qwen2.5VL-3B(2A), which uses only past actions, with Qwen2.5VL-3B(2AO), which includes both actions and observations from the past two steps. Incorporating visual history brings clear improvements across all metrics (+8.96% SR), highlighting the importance of visual context in guiding decision-making. When applying inference-only compression to the 2AO model, performance drops significantly (-4.95% SR). In contrast, our **Hicon-Agent-3B**, trained with history-aware optimization, recovers most of this loss and improves performance(+5.06 % SR) compared to the compressed baseline. It also exceeds the uncompressed 2AO baseline in SR and grounding accuracy, demonstrating more effective and efficient usage of historical context. This highlights the advantage of our training

strategy in mitigating the trade-off between computational efficiency and task performance.

Models	Hist.	Comp.	Type	Ground.	SR	FLOPs
Qwen2.5VL-3B	2A	–	61.05	59.61	43.33	13.21
Qwen2.5VL-3B	2AO	–	69.29	63.88	52.29	35.75
Qwen2.5VL-3B	2AO	✓	66.56(-2.73)	61.10(-2.78)	47.34(-4.95)	25.21(-10.54)
Hicon-Agent-3B	2AO	✓	67.79(+1.23)	65.01(+3.91)	52.40(+5.06)	25.21(-10.54)

Table 6. Study on different strategies for history utilization on the AndroidControl dataset. **Red** indicates improvement, **green** indicates drop. **Hist.** denotes the history context format, and **Comp.** indicates whether history compression is applied.

E. Algorithm details

Algorithm 1 details the HCPO training loop. We begin with on-policy, group-wise rollouts using Dynamic Context Sampling: for each of the G samples, we first construct the full input context $\{I, H_t, s_t\}$, then draw a truncated history H_t^i by sampling a history length $\tau_i \leq \tau$ sampled from the distribution p . Given this truncated context, we sample a response $o_i \sim \pi_{\theta_{\text{old}}}(\cdot | I, H_t^i, s_t)$. The corresponding reward $\{r_i\}$ is computed and converted into group advantages $\{A_i\}$.

Each sampled response is then evaluated by two forward passes of the current policy with shared parameters: (i) an uncompressed branch that uses $\{I, H_t, s_t\}$ end-to-end; and (ii) a compressed branch that mirrors the first k layers and then drops history *vision* tokens after layer k , retaining action and other tokens to form $H_t^{i,c}$. Reusing the same $\{o_i\}$ isolates the effect of compression.

We optimize two clipped-ratio gradient policy losses against π_{θ} , while adding a token-level consistency term that pulls the compressed distribution $\pi_{\theta}(\cdot | I, H_t^{i,c}, s_t)$ toward the uncompressed distribution $\pi_{\theta}(\cdot | I, H_t^i, s_t)$. Teacher logits from the uncompressed branch are detached to prevent gradient flow. The final objective L_{HCPO} preserves on-policy learning under complete history and aligns the compressed

path for efficient inference.

Algorithm 1 History Context-aware Policy Optimization (HCPO)

Require: Policy model π_θ , old policy $\pi_{\theta_{\text{old}}}$, reward model R , task instruction I , current GUI observation s_t , history context $H_t = \{(s_{t-\tau}, a_{t-\tau}), \dots, (s_{t-1}, a_{t-1})\}$, group size G , compression layer k , consistency weight λ

- 1: **# Group rollout with DCS**
 - 2: Initialize sampling distribution p
 - 3: Build full history sequence: $\{I, H_t, s_t\}$
 - 4: **for** $i = 1$ to G **do**
 - 5: Sample history length $\tau_i \sim p$ to get H_t^i
 - 6: Sample and select response $o_i \sim \pi_{\theta_{\text{old}}}(\cdot | I, H_t^i, s_t)$
 - 7: **end for**
 - 8: Evaluate rewards $\{r_1, \dots, r_G\} \leftarrow R(\{o_1, \dots, o_G\})$
 - 9: Compute group-wise advantages $\{A_1, \dots, A_G\}$
 - 10: **# Full history branch forward pass**
 - 11: **for** $i = 1$ to G **do**
 - 12: Compute logits $\pi_{\theta_{\text{old}}}(o_i | I, H_t^i, s_t)$
 - 13: Compute logits $\pi_\theta(o_i | I, H_t^i, s_t)$
 - 14: Compute reference logits $\pi_{\theta_{\text{ref}}}(o_i | I, H_t^i, s_t)$
 - 15: **end for**
 - 16: **# Compressed history branch forward pass**
 - 17: **for** $i = 1$ to G **do**
 - 18: Reuse response o_i and perform forward pass with partial history compression:
 - 19: Use uncompressed input $\{I, H_t^i, s_t\}$ in first k layers
 - 20: Drop history vision tokens from H_t^i in layers $k + 1$ to get $H_t^{i,c}$
 - 21: Compute compressed logits $\pi_\theta(o_i | I, H_t^{i,c}, s_t)$
 - 22: Compute token-level KL divergence: $\mathbb{D}_{\text{KL}}(\pi_\theta(o_i | I, H_t^{i,c}, s_t) \parallel \pi_\theta(o_i | I, H_t^i, s_t))$
 - 23: **end for**
 - 24: **# Compute policy-gradient loss and consistency loss**
 - 25: Compute uncompressed policy loss $\mathcal{L}_{\text{w/o comp}}$
 - 26: Compute compressed policy loss $\mathcal{L}_{\text{w/ comp}}$
 - 27: Compute consistency loss: $\mathcal{L}_{\text{KL}} = \sum_{i=1}^G \mathbb{D}_{\text{KL}}(\cdot)$
 - 28: Compute total loss: $\mathcal{L}_{\text{HCPO}} = \mathcal{L}_{\text{w/o comp}} + \mathcal{L}_{\text{w/ comp}} + \lambda \mathcal{L}_{\text{KL}}$
 - 29: Update model: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{HCPO}}$
-

F. Prompts for training and evaluation

You are a GUI agent. You are given a task and your action history, with screenshots. You need to perform the next action to complete the task. You FIRST need to think based on the current image, task, and historical

actions. The reasoning process MUST BE enclosed within **<think>** **</think>** tags. Then output the action, which MUST BE put in **<action>** **</action>** and MUST BE in Action Space.

```
## Output Format
<think>...</think><action>...</action>

## Action Space
click(start_box='(x,y)')
type(content='')
scroll(direction='down or up or right or left')
press_back()
press_home()
press_enter()
finished()

## Example:
<think>The user wants to search for shoes. The current screen has a search bar at the top.</think>
<action>click(start_box='(x,y)')</action>
```

Listing 1. AMEX training prompt template.

You are a GUI agent. You are given a task and your action history, with screenshots. You need to perform the next action to complete the task. You FIRST need to think based on the current image, task, and historical actions. The reasoning process MUST BE enclosed within **<think>** **</think>** tags. Then output the action, which MUST BE put in **<action>** **</action>** and MUST BE in Action Space.

```
## Output Format
<think>...</think><action>...</action>

## Action Space
click(start_box='(x,y)')
long_press(start_box='(x,y)')
type(content='')
scroll(direction='down or up or right or left')
open_app(app_name='')
press_back()
press_home()
wait()
finished()

## Example:
<think>The user wants to search for
```

```
shoes. The current screen has a search
bar at the top.</think>
<action>click(start_box=' (x,y)') </
action>
```

Listing 2. AndroidControl evaluation prompt template.

You are a GUI agent. You are given a task and your action history, with screenshots. You need to perform the next action to complete the task. You FIRST need to think based on the current image, task, and historical actions. The reasoning process MUST BE enclosed within `<think>` `</think>` tags. Then output the action, which MUST BE put in `<action>` `</action>` and MUST BE in Action Space.

```
## Output Format
<think>...</think><action>...</action>

## Action Space
click(start_box=' (x,y)')
long_press(start_box=' (x,y)')
type(content='')
scroll(direction='down or up or right
or left')
impossible()
press_back()
press_home()
press_recent()
finished()
## Example:
<think>The user wants to search for
shoes. The current screen has a search
bar at the top.</think>
<action>click(start_box=' (x,y)') </
action>
```

Listing 3. GUI-Odyssey evaluation prompt template.

You are a GUI agent. You are given a task and your action history, with screenshots. You need to perform the next action to complete the task. You FIRST need to think based on the current image, task, and historical actions. The reasoning process MUST BE enclosed within `<think>` `</think>` tags. Then output the action, which MUST BE put in `<action>` `</action>` and MUST BE in Action Space.

```
## Output Format
<think>...</think><action>...</action>
```

```
## Action Space
click(start_box=' (x,y)')
long_press(start_box=' (x,y)')
type(content='')
scroll(direction='down or up or right
or left')
impossible()
press_enter()
press_back()
press_home()
finished()
## Example:
<think>The user wants to search for
shoes. The current screen has a search
bar at the top.</think>
<action>click(start_box=' (x,y)') </
action>
```

Listing 4. AITW evaluation prompt template.

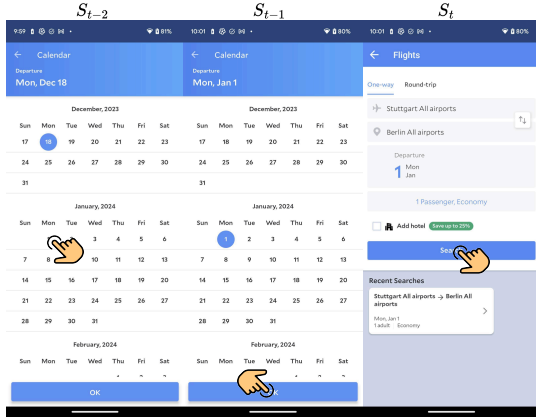
G. Model Behavior Across Different History Lengths

To better understand how history length affects agent behavior, we provide a case study comparing the base model and our HiconAgent-3B under different history lengths $\tau \in \{0, 1, 2\}$. As shown in Figure 10, the base model performs correctly when using shorter contexts ($\tau = 0$ or $\tau = 1$), but fails when the history is extended to $\tau = 2$, where the additional observations introduce distracting or misleading information, causing the model to attend to an incorrect UI element and produce the wrong action. In contrast, our model, trained with Dynamic Context Sampling, still produces the correct action when $\tau = 2$. Since DCS exposes the agent to diverse and progressively biased history lengths during optimization, the model learns to effectively utilize extended context. This qualitative evidence supports our quantitative results, demonstrating that naively increasing history is suboptimal, whereas HCPO equips the agent with robustness across variable context windows and enables it to benefit from longer history when necessary.

H. Visualization of successful trajectories

To better illustrate how our proposed HCPO framework facilitates robust decision-making, we visualize several representative successful trajectories from the evaluation benchmarks as shown in Figure 11a and Figure 11b.

Instruction: I'm going to meet my son in Berlin for important work with my wife and I decided to go there by flight, so search for the one way flight from Stuttgart to Berlin airport on January 1, 2024 in the Agoda app



Input Context: $\tau = 2 \{S_{t-2}, S_{t-1}, S_t\}$

Hicon-Agent-3B: <think>Since the goal is to search for a one-way flight from Stuttgart to Berlin, I need to click on the 'Search' button to find available flights from Stuttgart to Berlin on January 1, 2024. </think>
<action>click(start_box='(209,421)')</action>

Qwen2.5VL-3B: <think>The user wants to go from Stuttgart to Berlin, the next logical step is to click on the 'One-way' option to switch the search from round-trip to one-way. </think>
<action>click(start_box='(51,151)')</action>

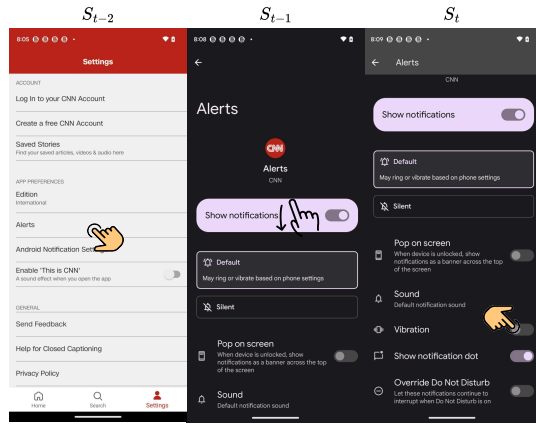
Input Context: $\tau = 1 \{S_{t-1}, S_t\}$

Qwen2.5VL-3B: <think>Since the departure and destination are already set, the next logical step is to initiate the search for flights. The 'Search' button is visible and ready to be clicked to proceed with the search. </think>
<action>click(start_box='(218,534)')</action>

Input Context: $\tau = 0 \{S_t\}$

Qwen2.5VL-3B: <think>The current screen is already set up with the correct departure and destination airports. The next logical step is to initiate the search for available flights. The 'Search' button is visible and ready to be clicked. </think>
<action>click(start_box='(213,534)')</action>

Instruction: In the CNN App, turn on the breaking news alert with vibration to receive an alert in Settings.



Input Context: $\tau = 2 \{S_{t-2}, S_{t-1}, S_t\}$

Hicon-Agent-3B: <think>Since the goal is to enable the vibration alert, I need to enable the Vibration option to complete the task of setting up the alert with pop-up and vibration. </think>
<action>click(start_box='(375,705)')</action>

Qwen2.5VL-3B: <think>The user wants to turn on the breaking news alert with vibration, and the vibration option is already enabled, the task is complete. The user has successfully configured the necessary setting. </think>
<action>finished()</action>

Input Context: $\tau = 1 \{S_{t-1}, S_t\}$

Qwen2.5VL-3B: <think>The goal is to turn on the breaking news alert with vibration, I need to enable it by clicking on the toggle switch. This will allow the user to receive an alert with a vibration. </think>
<action>click(start_box='(373,708)')</action>

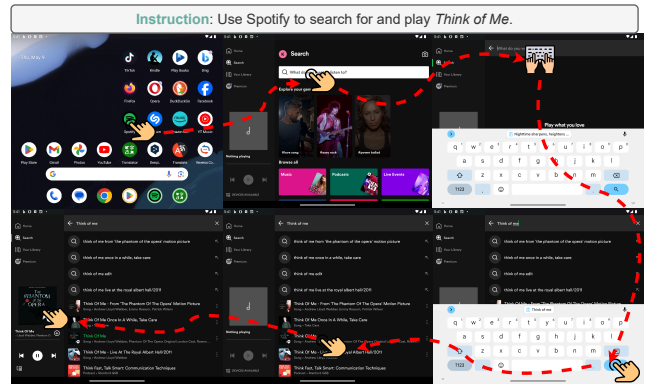
Input Context: $\tau = 0 \{S_t\}$

Qwen2.5VL-3B: <think>I need to ensure that the 'Vibration' toggle is switched on. The current screen shows the 'Vibration' toggle is off, so I need to click on it to enable it. </think>
<action>click(start_box='(375,711)')</action>

Figure 10. Case study of model behavior under different history length context.



(a) Trajectory example 1



(b) Trajectory example 2

Figure 11. Case studies of our model on downstream GUI navigation tasks.